



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 4

Dr. Asad Mahmood

Jan. 30, 2026

Lecture Outline

- Announcements (Assignment and Quiz 1, Collective Quiz arrangement)
- Outline of Previous Lecture:
 - **Chapter 1 - Computer Abstractions and Technology**
 - 1.7 The Power Wall
 - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
 - 1.9 Real Stuff: Benchmarking the Intel Core i7
 - 1.10 Going Faster: Matrix Multiply in Python
 - 1.11 Fallacies and Pitfalls
 - 1.12 Concluding Remarks
- Outline to Today's Lecture:
 - **Chapter 2 - Instructions: Language of the Computer**
 - 2.1 Introduction
 - 2.2 Operations of the Computer Hardware
 - 2.3 Operands of the Computer Hardware

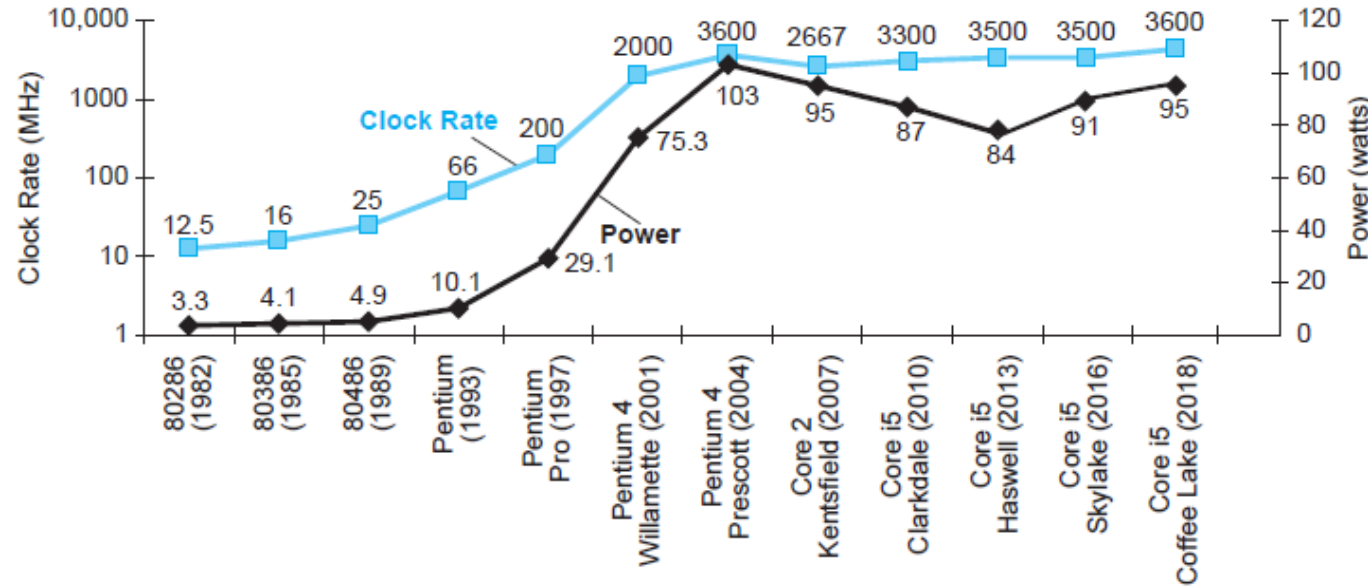
Chapter 1

Computer Abstractions and Technology

Chapter Outline

- **1 Computer Abstractions and Technology**
 - 1.1 Introduction
 - 1.2 Seven Great Ideas in Computer Architecture
 - 1.3 Below Your Program
 - 1.4 Under the Covers
 - 1.5 Technologies for Building Processors and Memory
 - 1.6 Performance
 - [1.7 The Power Wall](#)
 - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
 - 1.9 Real Stuff: Benchmarking the Intel Core i7
 - 1.10 Going Faster: Matrix Multiply in Python
 - 1.11 Fallacies and Pitfalls
 - 1.12 Concluding Remarks
 - 1.13 Historical Perspective and Further Reading

Power Trends



- In CMOS IC technology

$$\text{Power} = \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency}$$

×30

5V → 1V

×1000

Reducing Power

- Suppose a new CPU has
 - 85% of capacitive load of old CPU
 - 15% voltage and 15% frequency reduction

$$\frac{P_{\text{new}}}{P_{\text{old}}} = \frac{C_{\text{old}} \times 0.85 \times (V_{\text{old}} \times 0.85)^2 \times F_{\text{old}} \times 0.85}{C_{\text{old}} \times V_{\text{old}}^2 \times F_{\text{old}}} = 0.85^4 = 0.52$$

- The power wall
 - We can't reduce voltage further
 - We can't remove more heat
- How else can we improve performance?

Chapter Outline

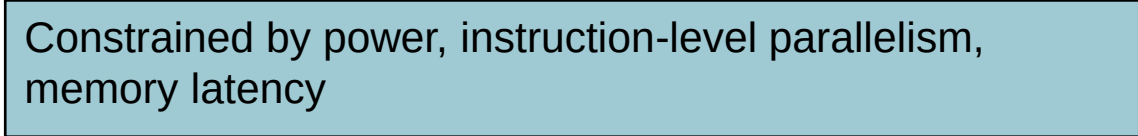
- **1 Computer Abstractions and Technology**
 - 1.1 Introduction
 - 1.2 Seven Great Ideas in Computer Architecture
 - 1.3 Below Your Program
 - 1.4 Under the Covers
 - 1.5 Technologies for Building Processors and Memory
 - 1.6 Performance
 - 1.7 The Power Wall
 - **1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors**
 - 1.9 Real Stuff: Benchmarking the Intel Core i7
 - 1.10 Going Faster: Matrix Multiply in Python
 - 1.11 Fallacies and Pitfalls
 - 1.12 Concluding Remarks
 - 1.13 Historical Perspective and Further Reading

1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors

“Up to now, most software has been like music written for a solo performer; with the current generation of chips we’re getting a little experience with duets and quartets and other small ensembles; but scoring a work for large orchestra and chorus is a different kind of challenge.”

Brian Hayes, *Computing in a Parallel Universe*, 2007.

§1.8 The Sea Change: The Switch to Multiprocessors



Multiprocessors

- Multicore microprocessors
 - More than one processor per chip
- Requires explicitly parallel programming
 - Compare with instruction level parallelism
 - Hardware executes multiple instructions at once
 - Hidden from the programmer
 - Programming for multicore -> Hard to do
 - Programming for performance/speed
 - Load balancing
 - Optimizing communication and synchronization

Chapter Outline

- **1 Computer Abstractions and Technology**
 - 1.1 Introduction
 - 1.2 Seven Great Ideas in Computer Architecture
 - 1.3 Below Your Program
 - 1.4 Under the Covers
 - 1.5 Technologies for Building Processors and Memory
 - 1.6 Performance
 - 1.7 The Power Wall
 - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
 - [1.9 Real Stuff: Benchmarking the Intel Core i7](#)
 - 1.10 Going Faster: Matrix Multiply in Python
 - 1.11 Fallacies and Pitfalls
 - 1.12 Concluding Remarks
 - 1.13 Historical Perspective and Further Reading

SPEC CPU Benchmark

- Programs used to measure performance
 - Supposedly typical of actual workload
- Standard Performance Evaluation Corp (SPEC)
 - Develops benchmarks for CPU, I/O, Web, ...
- SPEC CPU2006
 - Elapsed time to execute a selection of programs
 - Negligible I/O, so focuses on CPU performance
 - Normalize relative to reference machine
 - Summarize as geometric mean of performance ratios
 - CINT2006 (integer) and CFP2006 (floating-point)

$$\sqrt[n]{\prod_{i=1}^n \text{Execution time ratio}_i}$$

SPECspeed 2017 Integer benchmarks on a 1.8 GHz Intel Xeon E5-2650L

<i>Description</i>	<i>Name</i>	<i>Instruction Count x 10⁹</i>	<i>CPI</i>	<i>Clock cycle time (seconds x 10⁻⁹)</i>	<i>Execution Time (seconds)</i>	<i>Reference Time (seconds)</i>	<i>SPECratio</i>
Perl interpreter	perlbench	2684	0.42	0.556	627	1774	2.83
GNU C compiler	gcc	2322	0.67	0.556	863	3976	4.61
Route planning	mcf	1786	1.22	0.556	1215	4721	3.89
Discrete Event simulation - computer network	omnetpp	1107	0.82	0.556	507	1630	3.21
XML to HTML conversion via XSLT	xalancbmk	1314	0.75	0.556	549	1417	2.58
Video compression	x264	4488	0.32	0.556	813	1763	2.17
Artificial Intelligence: alpha-beta tree search (Chess)	deepsjeng	2216	0.57	0.556	698	1432	2.05
Artificial Intelligence: Monte Carlo tree search (Go)	leela	2236	0.79	0.556	987	1703	1.73
Artificial Intelligence: recursive solution generator (Sudoku)	exchange2	6683	0.46	0.556	1718	2939	1.71
General data compression	xz	8533	1.32	0.556	6290	6182	0.98
Geometric mean							2.36

SPEC Power Benchmark

- Given the increasing importance of energy and power, SPEC added a benchmark to measure power. at different workload levels
- SPECpower_ssj2008 for Xeon E5-2650L

Target Load %	Performance (ssj_ops)	Average Power (watts)
100%	4,864,136	347
90%	4,389,196	312
80%	3,905,724	278
70%	3,418,737	241
60%	2,925,811	212
50%	2,439,017	183
40%	1,951,394	160
30%	1,461,411	141
20%	974,045	128
10%	485,973	115
0%	0	48
Overall Sum	26,815,444	2,165
$\Sigma \text{ssj_ops} / \Sigma \text{power} =$		12,385

Chapter Outline

- **1 Computer Abstractions and Technology**
 - 1.1 Introduction
 - 1.2 Seven Great Ideas in Computer Architecture
 - 1.3 Below Your Program
 - 1.4 Under the Covers
 - 1.5 Technologies for Building Processors and Memory
 - 1.6 Performance
 - 1.7 The Power Wall
 - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
 - 1.9 Real Stuff: Benchmarking the Intel Core i7
 - [1.10 Going Faster: Matrix Multiply in Python](#)
 - 1.11 Fallacies and Pitfalls
 - 1.12 Concluding Remarks
 - 1.13 Historical Perspective and Further Reading

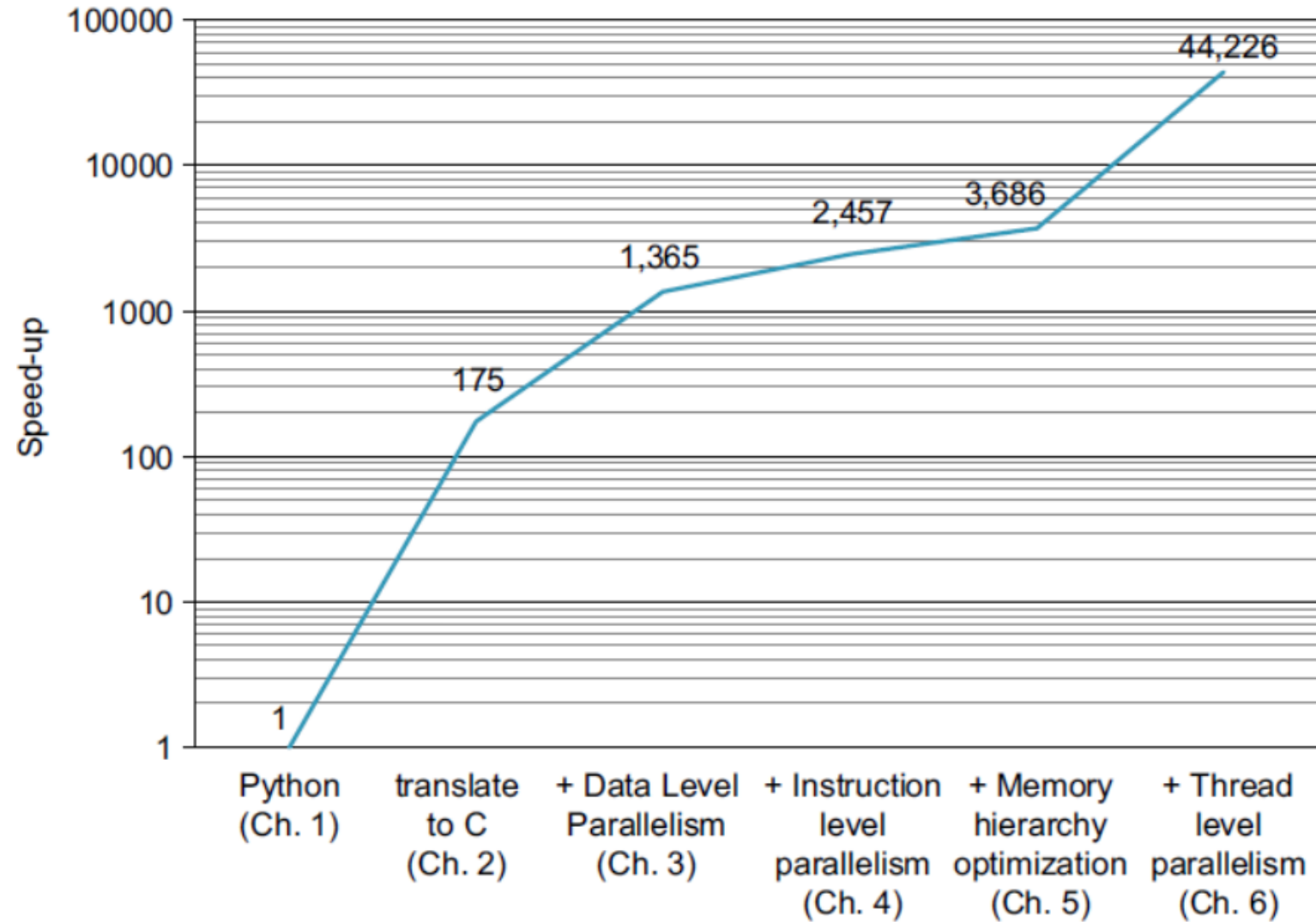
Going Faster: Matrix Multiply in Python

- To show the impact of different strategies in this book to improve performance, a matrix multiplication example is used with same Hardware (n1-standard-96 server in Google Cloud Engine, which has two Intel Skylake Xeon chips, and each chip has 24 processors or cores)

```
for i in xrange(n):  
    for j in xrange(n):  
        for k in xrange(n):  
            C[i][j] += A[i][k] * B[k][j]
```

- When Python code is used
 - For matrix size 960*960 -> 5 minutes
 - For matrix size 4096*4096 -> 6 hours!

Going Faster: Matrix Multiply in Python



Chapter Outline

- **1 Computer Abstractions and Technology**
 - 1.1 Introduction
 - 1.2 Seven Great Ideas in Computer Architecture
 - 1.3 Below Your Program
 - 1.4 Under the Covers
 - 1.5 Technologies for Building Processors and Memory
 - 1.6 Performance
 - 1.7 The Power Wall
 - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
 - 1.9 Real Stuff: Benchmarking the Intel Core i7
 - 1.10 Going Faster: Matrix Multiply in Python
 - [1.11 Fallacies and Pitfalls](#)
 - 1.12 Concluding Remarks
 - 1.13 Historical Perspective and Further Reading

Fallacies and Pitfalls

- “*Science must begin with myths, and the criticism of myths*” - Sir Karl Popper, *The Philosophy of Science*, 1957
- Each chapter of this book has some **fallacies** and **pitfalls** mentioned at the end to clear misconceptions.
 - **Fallacy**: commonly held mis-conceptions
 - **Pitfall**: Mistake e.g. generalization of a concept which is only true in a limited context

Pitfall

- *Pitfall: Improving an aspect of a computer and expecting a proportional improvement in overall performance*

Amdahl's Law

$$T_{\text{improved}} = \frac{T_{\text{affected}}}{\text{improvement factor}} + T_{\text{unaffected}}$$

- **Example:** If a program takes 100s to run, how much overall improvement in execution time for a program do we get if we reduce the multiplication time by 50%. Consider the cases of:
 - Case 1: Multiplications take 80s out of 100s
 - Case 2: Multiplications take 20s out of 100s

Fallacy

- *Fallacy: Computers at low utilization use little power.*
- Look back at i7 power benchmark
 - At 100% load: 258W
 - At 50% load: 170W (66%)
 - At 10% load: 121W (47%)
- Google data center
 - Mostly operates at 10% – 50% load
 - At 100% load less than 1% of the time
- Consider designing processors to make power proportional to load

Pitfall

- *Pitfall: Using a subset of the performance equation as a performance metric.*
- MIPS: Millions of Instructions Per Second
 - Doesn't account for
 - Differences in ISAs between computers
 - Differences in complexity between instructions
- Example:
 - Which computer has higher MIPS?
 - Which computer is faster?

Measurement	Computer A	Computer B
Instruction count	10 billion	8 billion
Clock rate	4 GHz	4 GHz
CPI	1.0	1.1

Chapter Outline

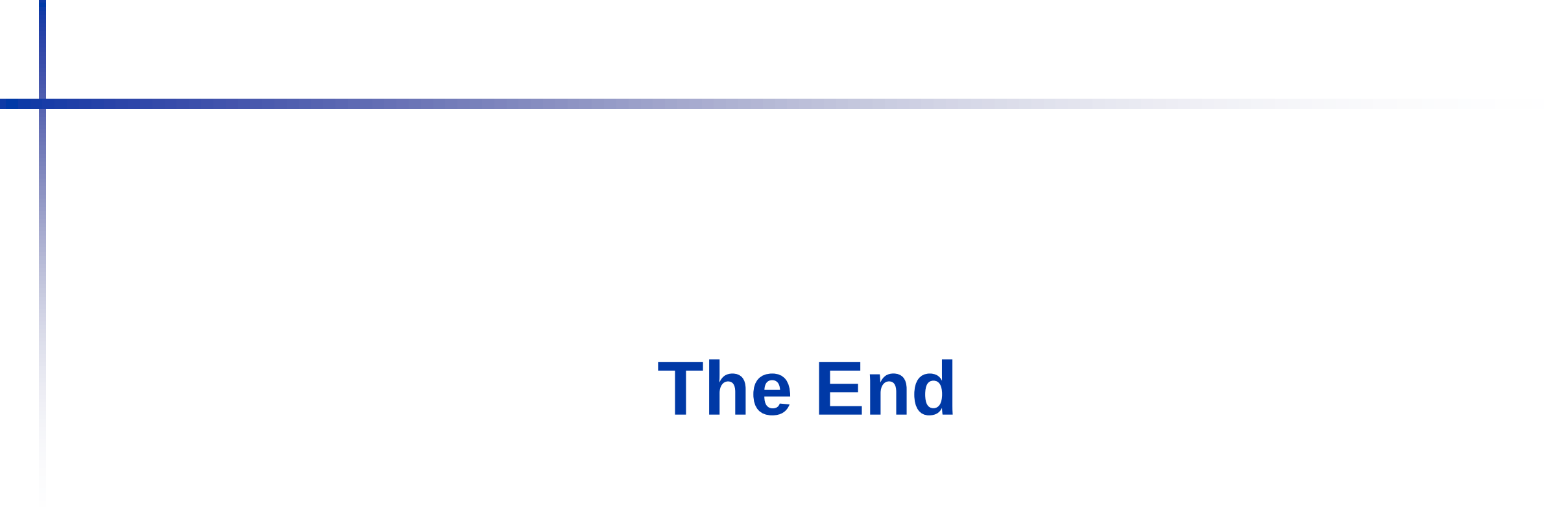
- **1 Computer Abstractions and Technology**
 - 1.1 Introduction
 - 1.2 Seven Great Ideas in Computer Architecture
 - 1.3 Below Your Program
 - 1.4 Under the Covers
 - 1.5 Technologies for Building Processors and Memory
 - 1.6 Performance
 - 1.7 The Power Wall
 - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
 - 1.9 Real Stuff: Benchmarking the Intel Core i7
 - 1.10 Going Faster: Matrix Multiply in Python
 - 1.11 Fallacies and Pitfalls
 - [1.12 Concluding Remarks](#)
 - 1.13 Historical Perspective and Further Reading

Concluding Remarks

- Cost/performance is improving
 - Due to underlying technology development
- Hierarchical layers of abstraction
 - In both hardware and software
- Instruction set architecture
 - The hardware/software interface
- Execution time: the best performance measure
- Power is a limiting factor
 - Use parallelism to improve performance

Chapter Outline

- **1 Computer Abstractions and Technology**
 - 1.1 Introduction
 - 1.2 Seven Great Ideas in Computer Architecture
 - 1.3 Below Your Program
 - 1.4 Under the Covers
 - 1.5 Technologies for Building Processors and Memory
 - 1.6 Performance
 - 1.7 The Power Wall
 - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
 - 1.9 Real Stuff: Benchmarking the Intel Core i7
 - 1.10 Going Faster: Matrix Multiply in Python
 - 1.11 Fallacies and Pitfalls
 - 1.12 Concluding Remarks
 - **1.13 Historical Perspective and Further Reading –Self-Study**



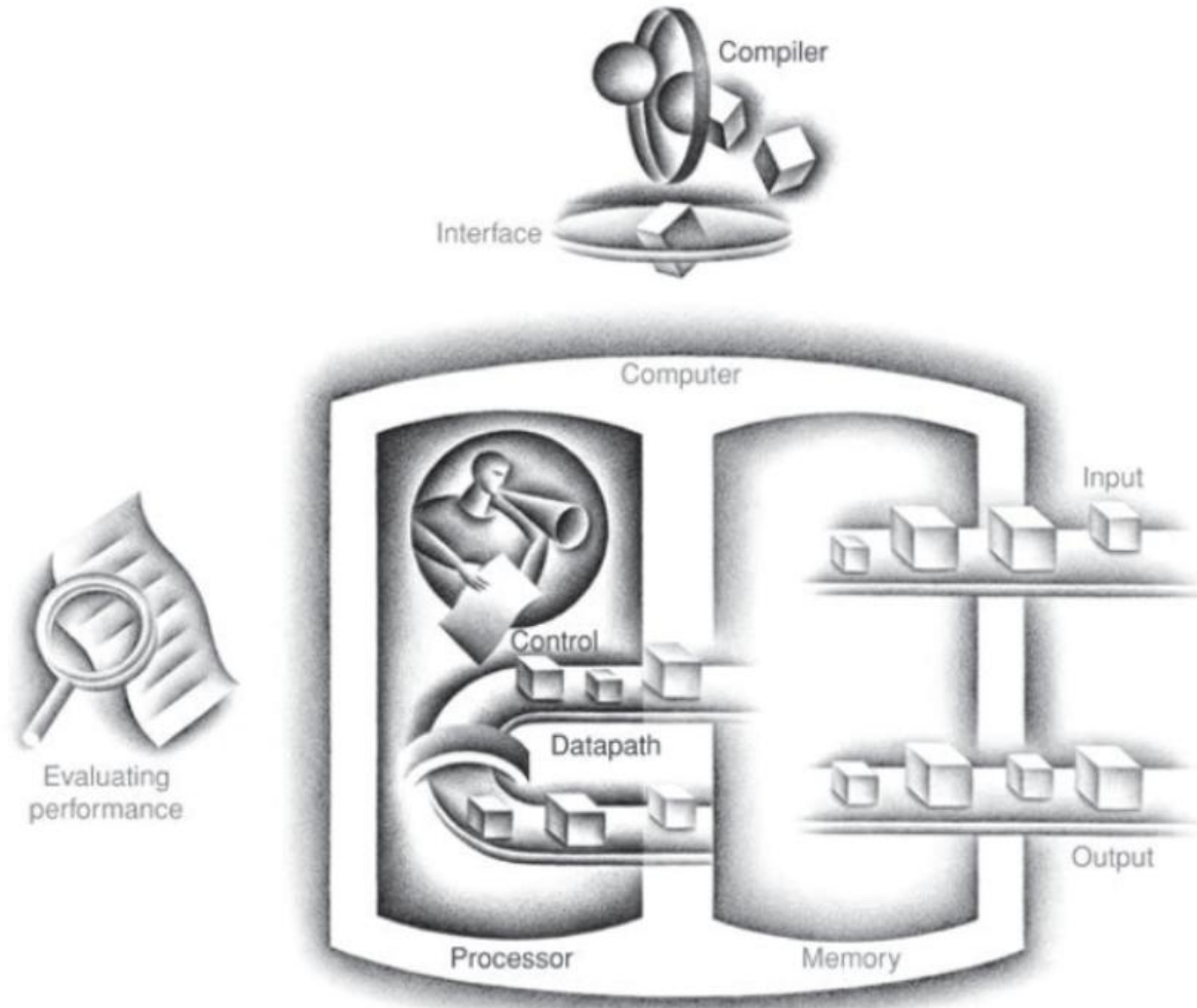
The End

Chapter 2

Instructions: Language of the Computer

Chapter Theme

The Five Classic Components of a Computer



Chapter Outline

- **2 Instructions: Language of the Computer**
 - [2.1 Introduction](#)
 - 2.2 Operations of the Computer Hardware
 - 2.3 Operands of the Computer Hardware
 - 2.4 Signed and Unsigned Numbers
 - 2.5 Representing Instructions in the Computer
 - 2.6 Logical Operations
 - 2.7 Instructions for Making Decisions
 - 2.8 Supporting Procedures in Computer Hardware
 - 2.9 Communicating with People
 - 2.10 RISC-V Addressing for Wide Immediates and Addresses
 - 2.11 Parallelism and Instructions: Synchronization
 - 2.12 Translating and Starting a Program
 - 2.13 A C Sort Example to Put it All Together
 - 2.14 to 2.24

Introduction

- To command a computer's hardware, you must speak its language
 - The language of computer is a set of instructions called as the instruction set.
- Different computers have different instruction sets
 - But with many aspects in common
 - Examples: RISC-V, MIPS, x86 etc.
- Many computers have a very simple instruction sets
 - Simplified implementation
- Instruction Set Architecture (ISA) = Instruction Set + rules + data types, addressing modes etc.
- ISA is a consequence of the Stored-Program concept.

Chapter Outline

- **2 Instructions: Language of the Computer**
 - 2.1 Introduction
 - **2.2 Operations of the Computer Hardware**
 - 2.3 Operands of the Computer Hardware
 - 2.4 Signed and Unsigned Numbers
 - 2.5 Representing Instructions in the Computer
 - 2.6 Logical Operations
 - 2.7 Instructions for Making Decisions
 - 2.8 Supporting Procedures in Computer Hardware
 - 2.9 Communicating with People
 - 2.10 RISC-V Addressing for Wide Immediates and Addresses
 - 2.11 Parallelism and Instructions: Synchronization
 - 2.12 Translating and Starting a Program
 - 2.13 A C Sort Example to Put it All Together
 - 2.14 to 2.24

The RISC-V Instruction Set

- Used as the example throughout the book
- Developed at UC Berkeley as open ISA
- Now managed by the RISC-V Foundation (riscv.org)
- Similar ISAs have a large share of embedded core market
 - Applications in consumer electronics, network/storage equipment, cameras, printers, ...

Arithmetic Operations

- Add and subtract, three **operands**
 - Two sources and one destination
add a, b, c // a gets b + c
- All arithmetic operations have this form
- *Design Principle 1: Simplicity favors regularity*
 - Regularity makes implementation simpler
 - Simplicity enables higher performance at lower cost

RISC-V Operands and Assemble Language

Name	Example	Comments		
32 registers	x0 - x31	Fast locations for data. In RISC-V, data must be in registers to perform arithmetic. Register x0 always equals 0.		
2 ³⁰ memory words	Memory[0], Memory[4], ..., Memory[4,294,967,292]	Accessed only by data transfer instructions. RISC-V uses byte addresses, so sequential word accesses differ by 4. Memory holds data structures, arrays, and spilled registers.		

Category	Instruction	Example	Meaning	Comments
Arithmetic	Add	add x5, x6, x7	$x5 = x6 + x7$	Three register operands; add
	Subtract	sub x5, x6, x7	$x5 = x6 - x7$	Three register operands; subtract
	Add immediate	addi x5, x6, 20	$x5 = x6 + 20$	Used to add constants
	Load word	lw x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	Word from memory to register
Logical	And	and x5, x6, x7	$x5 = x6 \& x7$	Three reg. operands; bit-by-bit AND
	Inclusive or	or x5, x6, x8	$x5 = x6 \mid x8$	Three reg. operands; bit-by-bit OR
	Exclusive or	xor x5, x6, x9	$x5 = x6 \wedge x9$	Three reg. operands; bit-by-bit XOR
	And immediate	andi x5, x6, 20	$x5 = x6 \& 20$	Bit-by-bit AND reg. with constant
Data transfer	Load word	lw x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	Word from memory to register
	Load word, unsigned	lwu x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	Unsigned word from memory to register
	Store word	sw x5, 40(x6)	$\text{Memory}[x6 + 40] = x5$	Word from register to memory
	Load halfword	lh x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	Halfword from memory to register
	Load halfword, unsigned	lhu x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	Unsigned halfword from memory to register

Arithmetic Example

- C code:

$a = b + c;$

$d = a - e;$

- Compiled RISC-V code?

Arithmetic Example

- C code:

$$f = (g + h) - (i + j);$$

- Compiled RISC-V code?

Chapter Outline

- **2 Instructions: Language of the Computer**
 - 2.1 Introduction
 - 2.2 Operations of the Computer Hardware
 - **2.3 Operands of the Computer Hardware**
 - 2.4 Signed and Unsigned Numbers
 - 2.5 Representing Instructions in the Computer
 - 2.6 Logical Operations
 - 2.7 Instructions for Making Decisions
 - 2.8 Supporting Procedures in Computer Hardware
 - 2.9 Communicating with People
 - 2.10 RISC-V Addressing for Wide Immediates and Addresses
 - 2.11 Parallelism and Instructions: Synchronization
 - 2.12 Translating and Starting a Program
 - 2.13 A C Sort Example to Put it All Together
 - 2.14 to 2.24

RISC-V Operands

- Three types of operand are commonly used in the RISC-V Instruction Set / Assembly Language:
 1. Register Operands
 2. Memory Operands
 3. Immediate Operands

Register Operands

- Arithmetic instructions use **register** operands
- Registers are **small-sized, fastly-accessed memory locations** very close to CPU
- RISC-V has a **32 × 64-bit register-file**
 - Use for frequently accessed data
 - **32-bit data is called a “word”**
 - 64-bit data is called a **“doubleword”**
 - 32 x 64-bit general purpose registers x0 to x31
- *Design Principle 2: Smaller is faster*
 - Larger register-file size may increase delay

RISC-V Registers

- x0: the constant value 0
- x1: return address
- x2: stack pointer
- x3: global pointer
- x4: thread pointer
- x5 – x7, x28 – x31: temporaries
- x8: frame pointer
- x9, x18 – x27: saved registers
- x10 – x11: function arguments/results
- x12 – x17: function arguments

Register Operand Example

- C code:

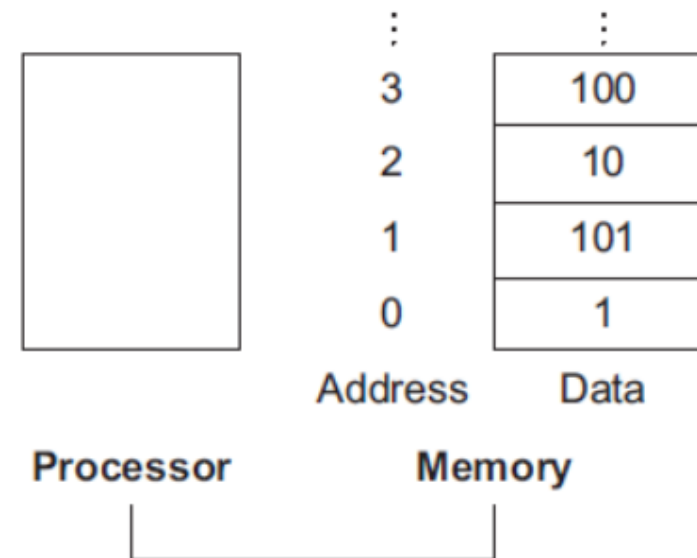
`f = (g + h) - (i + j);`

- `f, ..., j` in `x19, x20, ..., x23`

- Compiled RISC-V code?

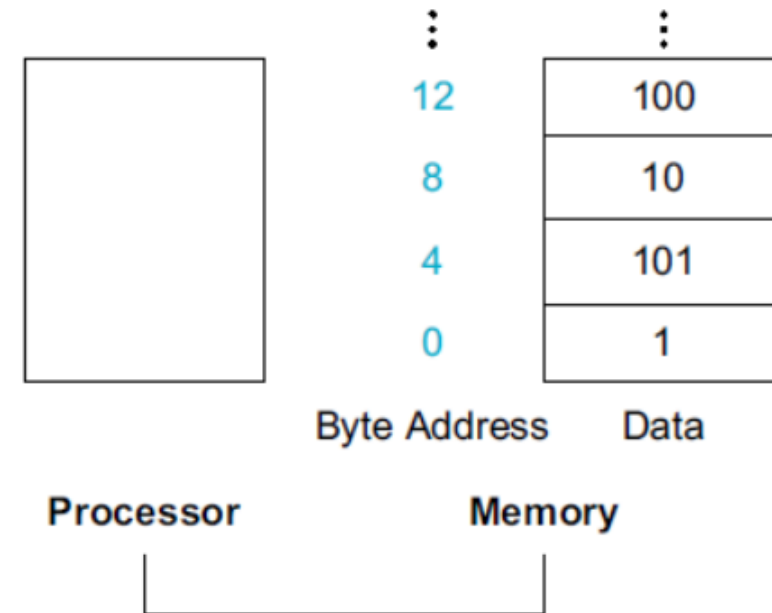
Memory Operands

- Main memory used to store more complex data-structures, e.g. Arrays, structures, dynamic data etc.
- To apply arithmetic operations
 - Data transfer instructions (load, store)
 - Load values from memory into registers
 - Store result from register to memory
- Memory is just a large, single-dimensional array, with the address acting as the index to that array, starting at 0.



Memory Operands

- Memory is byte addressed
 - Each address identifies an 8-bit byte
- RISC-V is Little Endian
 - Least-significant byte at least address of a word
 - *c.f.* Big Endian: most-significant byte at least address
- RISC-V does not require words to be aligned in memory
 - Unlike some other ISAs



Memory Operand Example

- Assume an array A of 100 words
- C code:
 $g = h + A[8];$
 - Variables g, h in registers x20 and x21 respectively
 - base address of A in register x22
- Compiled RISC-V code?